Work Breakdown (Sprint-Based Roadmap)

To manage development, we will follow an **agile methodology** with 10 sprints (approximately 2 weeks per sprint) covering February – June.

Each sprint produces specific deliverables, progressively building the system from initial setup to full deployment. The team of three students will split tasks such that each member leads certain modules while collaborating on integration.

Below is an overview of the sprints and their focus:

Sprint	Timeline	Focus & Key Deliverables
1	Feb 1–Feb 14	Project setup (repository, CI/CD, development environment)
2	Feb 15–Feb 28	Basic load balancer (classical routing) implemented and tested
3	Mar 1–Mar 14	Telemetry pipeline and metrics database in place
4	Mar 15–Mar 28	Data preparation: gather datasets and simulate environment for ML
5	Apr 1–Apr 14	Anomaly detection module implemented and integrated
6	Apr 15–Apr 28	Forecasting module and autoscaler implemented and integrated
7	May 1–May 14	Reinforcement learning engine integrated (shadow testing)
8	May 15–May 28	RL-driven routing fully enabled; policy manager integrated
9	Jun 1–Jun 14	System-wide testing, performance benchmarking, and tuning
10	Jun 15–Jun 30	Deployment, final optimizations, documentation, and presentation

Sprint 1: Initial Setup & Infrastructure (Feb 1–Feb 14)

Sprint Goal Establish a shared development foundation, including repository setup, CI/CD, containerized development, and a finalized baseline architecture.

Sprint Tasks

- **Repository & Version Control Setup**
  - Initialize the SmartLoad GitHub repository (public or private as appropriate)
  - Define project directory structure for each microservice/module:
    - `load-balancer/`
    - `rl-engine/`
    - `telemetry/`
  - Add a clear README describing the project and structure
  - Establish branching and merging conventions (feature branches, pull requests)
  - Ensure repository access for all team members
- **CI/CD Pipeline Configuration**
  - Set up continuous integration using GitHub Actions (or equivalent)
  - Configure workflows to:
    - Lint code on each push
    - Run a basic (initial) test suite
    - Build Docker images
  - Plan for future expansion of the pipeline to include module-level testing
- **Development Environment & Containerization**
  - Create Dockerfiles for each major component
  - Provide a `docker-compose.yml` to orchestrate services in development
  - Initially support a minimal setup (placeholder services)
  - Verify that a simple "Hello World" service (load balancer ping response) runs successfully via Docker
- **Baseline Architecture Finalization**
  - Review and confirm technology stack decisions (detailed in the next section)
  - Agree on:
    - Programming languages and frameworks per component

        * Responsibilities of each module
    - Walk through the **Modular Architecture Overview**
    - Define and document interfaces between components, including:
        * Control bus data
        * Telemetry message formats

- **Sprint Planning & Task Tracking**
    - Set up GitHub Projects or Issues for task management
    - Create and populate Sprint 1 tasks
    - Define clear acceptance criteria (CI passing, repo structure finalized)

Sprint Deliverables By the end of Sprint 1, the team will have:

- A fully initialized project repository with agreed structure
- A functioning CI pipeline with dummy tests passing
- Working containerized development environment
- A shared and documented understanding of the system architecture

Sprint completion will be marked by a **sprint review meeting**, during which the team demonstrates the environment setup and adjusts the project plan as needed.

Sprint 2: Basic Ingress & Load Balancing (Feb 15–Feb 28)

Sprint Goal Implement a functional traffic ingress and baseline load balancing capability to distribute HTTP requests across multiple backend servers.

Sprint Tasks

- **Load Balancer Service Implementation**
    - Develop the traffic ingress component that accepts client HTTP requests and forwards them to backend servers
    - Implement the service as a minimal HTTP reverse proxy:
        * Primary choice: Go using the built-in `net/http` library
        * Fallback option: Node.js with Express (if Go learning is delayed)
    - Implement **Round-Robin** routing as the initial load balancing policy

  - Configure the service to read backend server addresses from:
    * Configuration files, or
    * Environment variables

- **Dummy Backend Services**
  - Implement a lightweight backend service (Python/Flask or Node.js)
  - Return a fixed response to incoming requests
  - Optionally simulate processing latency (sleep for a few milliseconds)
  - Run multiple instances in separate containers to form the backend server pool

- **Baseline Health Check Mechanism**
  - Implement periodic health checks from the load balancer to the backend instances
  - Detect unresponsive backends and exclude them from request routing
  - Maintain simple round-robin behavior over only healthy instances
  - Lay the groundwork future anomaly detection integration

- **Logging & Basic Telemetry Collection**
  - Log each incoming request handled by the load balancer
  - Capture, at minimum:
    * Timestamp
    * Selected backend instance
    * Response time
  - Output telemetry to standard output or log files as an initial baseline
  - Prepare for later integration with the full telemetry pipeline

- **Unit Testing of Load Balancer Logic**
  - Write unit tests for core routing behavior, including:
    * Correct round-robin server selection
    * Proper handling of backend removal or failure
  - Integrate tests into the CI pipeline to ensure reliability

- **Code Review & Integration**
  - Conduct peer code reviews prior to merging into the main branch
  - Ensure adherence to quality, correctness, and shared understanding

Sprint Deliverables By the end of Sprint 2, the team will have:

- A containerized, functional HTTP load balancer service
- Support for distributing traffic across multiple backend instances using a classical round-robin algorithm
- Basic health checking and request-level telemetry

A demonstration will show three dummy backend servers receiving evenly distributed requests through the load balancer, forming the foundation for subsequent intelligent routing enhancements.

Sprint 3: Telemetry & Observability Pipeline (Mar 1–Mar 14)

Sprint Goal Establish a complete observability pipeline that captures, stores, and visualizes standardized telemetry data produced by the load balancer, forming the data foundation for future decision-making and ML modules.

Sprint Tasks

- **Load Balancer Instrumentation with OpenTelemetry**
  - Integrate the OpenTelemetry (OTel) SDK into the load balancer service
  - Instrument key metrics, including:
    - Request count
    - Request rate
    - Request latency (histograms)
    - CPU and memory usage (where applicable)
  - Use the OTel Go SDK (or equivalent) to ensure metrics are captured in a standardized and portable format
- **Metrics Storage Backend Setup**
  - Deploy **TimescaleDB** as the primary time-series metrics store (PostgreSQL-based)
  - Run TimescaleDB as a Docker container
  - Design and create database schemas for telemetry data, such as:
    - Timestamped latency measurements
    - Backend/server identifiers
  - Optionally deploy a Prometheus server for real-time metric scraping

  - If Prometheus is used, plan for exporting long-term data to TimescaleDB via a connector

- **Telemetry Collection Pipeline**
  - Deploy an OpenTelemetry Collector service (if required)
  - Configure telemetry pipelines with the following stages:
    * Input: OTLP or Prometheus scraping
    * Processing: batching and basic transformations
    * Output: TimescaleDB, Prometheus remote write, or file-based export
  - Ensure reliable delivery of metrics from the load balancer to storage

- **Distributed Tracing Setup**
  - Initialize OpenTelemetry tracing in the load balancer service
  - Optionally instrument dummy backend services for end-to-end tracing
  - Configure trace output to logs or a tracing backend (Jaeger)
  - Treat this task as optional if sprint time becomes constrained

- **Metrics Visualization (Initial)**
  - Deploy a lightweight visualization tool (Grafana)
  - Connect the dashboard to TimescaleDB (or Prometheus, if used)
  - Create an initial dashboard displaying:
    * Request rates over time
    * Latency distributions
  - Use visualization primarily to validate correct data ingestion

- **Telemetry Data Validation & Testing**
  - Generate synthetic load using tools such as `curl` or simple scripts
  - Verify that telemetry data appears correctly in the metrics database
  - Validate:
    * Metric resolution and accuracy
    * Correct labeling (server IDs, request paths)
  - Refine instrumentation as needed to ensure data usability for ML modules

Sprint Deliverables By the end of Sprint 3, SmartLoad will include:

- A fully operational observability pipeline
- Standardized telemetry emitted by the load balancer
- Centralized storage of metrics for both real-time and historical analysis
- Basic dashboards validating correct data collection

This sprint establishes the **Telemetry & Monitoring** layer of the system, providing the data foundation required for adaptive control and learning-based decision engines in subsequent sprints.

Sprint 4: Data Collection & Preparation (Mar 15–Mar 28)

Sprint Goal Collect, preprocess, and analyze workload and anomaly datasets, and prepare preliminary predictive and anomaly detection models to support future online learning and decision-making components.

Sprint Tasks

- **Workload Trace Data Acquisition**
  - Collect historical workload traces for forecasting model training
  - Primary datasets include:
    - Google Borg cluster traces
    - Alibaba cluster traces
  - Download datasets in CSV or Proto formats containing time-series resource usage information
  - Extract and preprocess a representative subset of the data suitable for model training and evaluation
  - Optionally collect additional datasets for diversity, such as:
    - BitBrains VM workload traces
    - Azure VM workload traces
- **Anomaly Detection Dataset Acquisition**
  - Obtain labeled time-series datasets for anomaly detection
  - Primary sources include:

    - Numenta Anomaly Benchmark (NAB)
    - Yahoo Service Machine Dataset (SMD)
  - Parse datasets into a unified format (CSV with timestamp, metric value, and anomaly label)
  - Prepare data for training and evaluation of anomaly detection models

- **Simulated Environment for Reinforcement Learning**
  - Investigate and prototype a simulated environment for RL training, where agents interact with a controllable system rather than a static dataset
  - Evaluate the following simulation approaches:
    - **Mininet:** Network emulation with virtual hosts and switches to simulate routing decisions and resulting latencies
    - **CloudSim:** Cloud datacenter simulator supporting virtual machines, task scheduling, and accelerated-time experiments
  - Select one simulation tool for initial RL experiments
  - If existing tools prove overly complex, implement a fallback **custom Python simulator**, for example:
    - Simulate N servers with stochastic service times
    - RL action: select a server
    - Reward: negative request latency

- **Preliminary Offline Model Training**
  - Train an initial **workload forecasting model** using one of the collected workload trace datasets
  - Example approaches include:
    - ARIMA
    - Prophet
  - Predict future load (one hour ahead) and evaluate performance using metrics such as MAPE or RMSE
  - Train or configure a preliminary **anomaly detection model** using NAB or Yahoo SMD data
  - Possible methods include:
    - Statistical techniques (moving average with thresholds)
    - Machine learning methods (Isolation Forest, LSTM autoencoder)
  - Validate the model's ability to detect injected or labeled anomalies

- **Dataset Documentation & Licensing Review**

  - Document all datasets used, including:
    * Data sources
    * Preprocessing steps
    * Feature selection and transformations
  - Compile documentation into a Data Preparation report for inclusion in the final project documentation
  - Verify dataset licensing and usage permissions, noting required attributions (Google traces under CC-BY, NAB under MIT license)

Sprint Deliverables By the end of Sprint 4, the team will have:

- Curated and documented workload and anomaly datasets
- Preprocessing scripts and exploratory analysis artifacts (plots, statistics)
- Preliminary offline-trained forecasting and anomaly detection models
- A selected (or prototyped) simulation environment for RL training

This sprint does not introduce a user-facing system feature, but it provides critical groundwork for the machine learning components.

Sprint review discussions will focus on insights gained from the data, observed workload patterns, anomaly characteristics, and the feasibility of predictive and learning-based control.

Sprint 5: Anomaly Detection Module Integration (Apr 1–Apr 14)

Sprint Goal Integrate an anomaly detection module into SmartLoad that identifies unhealthy backend instances in real time and enforces failure-aware routing decisions in the load balancer.

Sprint Tasks

- **Anomaly Detection Microservice Implementation**
  - Implement the anomaly detection engine as a standalone microservice
  - Use Python to leverage established ML libraries (scikit-learn or PyTorch)
  - Consume telemetry data by:
    * Subscribing to the telemetry stream, or
    * Periodically querying the metrics database

  - Start with a simple statistical baseline, such as:
    * Rolling averages and standard deviations
    * Threshold-based deviation detection for latency or error rate
  - Integrate a more advanced anomaly detection model from Sprint 4 (Isolation Forest or LSTM-based model) for improved accuracy

- **Real-Time Anomaly Signaling & Control Bus Integration**
  - Define how anomaly alerts are published within the Decision Layer
  - Emit per-node "health status" signals indicating normal, degraded, or unhealthy states
  - Implement signaling using one of the following approaches:
    * Preferred: lightweight pub/sub mechanism (Redis Pub/Sub or MQTT)
    * Fallback: status updates stored in the metrics database or an in-memory store
  - Favor asynchronous, event-driven communication for scalability and decoupling

- **Load Balancer Integration & Routing Constraints**
  - Modify the load balancer to consume anomaly signals via:
    * Subscription to the control bus, or
    * Polling the anomaly detection service API
  - Enforce routing rules such that:
    * Instances flagged as unhealthy are excluded from routing
    * Degraded instances are down-weighted in server selection
  - Ensure that anomaly-based constraints are applied **prior to** other routing policies
  - Define recovery logic for backend instances, such as:
    * Automatic reintegration once anomalies clear, or
    * Manual reset for safety

- **Anomaly Scenario Testing & Validation**
  - Simulate abnormal backend behavior, including:
    * Artificial slowdowns
    * Increased error rates or unresponsiveness
  - Verify that the anomaly detection service correctly flags problematic instances
  - Confirm that the load balancer avoids or deprioritizes flagged nodes
  - Replay labeled anomaly sequences from NAB or Yahoo SMD datasets as pseudo-live telemetry input

  - Measure end-to-end detection and mitigation latency, targeting response within one or two monitoring intervals

- **Anomaly Module Documentation**
  - Produce a concise design document describing:
    * Chosen anomaly detection approach
    * Model parameters and thresholds
    * Interfaces with other SmartLoad components
  - Prepare the document for inclusion in the final project report

Sprint Deliverables By the end of Sprint 5, SmartLoad will include:

- An operational anomaly detection microservice
- Real-time health status signaling integrated into the Decision Layer
- A load balancer that dynamically avoids unhealthy or degraded nodes

The sprint demonstration will show that when a backend instance becomes slow or unresponsive, SmartLoad detects the anomaly and reroutes traffic accordingly, improving overall system resiliency and fulfilling the failure-aware control objective of the platform.

Sprint 6: Forecasting Module & Autoscaler Integration (Apr 15–Apr 28)

Sprint Goal Integrate a predictive forecasting module and an autoscaling controller that proactively adjusts system capacity based on anticipated future load, closing the control-and-actuation loop in SmartLoad.

Sprint Tasks

- **Forecasting Microservice Implementation**
  - Implement the workload forecasting engine as a standalone microservice
  - Use Python to leverage time-series forecasting libraries (Prophet, statsmodels)
  - Periodically query the metrics database for historical load data, such as:
    * Request rates
    * CPU utilization
  - Generate short-term load predictions (next 5-minute or 1-hour interval)

  - Begin with a simple baseline model (last-hour average predicts next-hour load) to validate data flow
  - Integrate a more advanced forecasting model trained in Sprint 4 (ARIMA, Prophet, or LSTM) once the pipeline is verified
  - Output predicted load values or ranges via the control bus

- **Autoscaling Decision Logic**
  - Design and implement the **Autoscaler** as a separate module for clarity and modularity
  - Subscribe to forecast outputs published on the control bus
  - Use forecasted load (and optional auxiliary signals) to decide scaling actions
  - Implement initial rule-based logic, for example:
    * Scale up if predicted load exceeds current capacity by $X\%$
    * Scale down if predicted load remains below capacity by $Y\%$ for Z minutes
  - Encode conservative defaults to avoid oscillations or thrashing

- **Scaling Infrastructure Integration**
  - Implement mechanisms to execute scaling actions at the infrastructure level
  - For a local or simulated environment:
    * Launch new dummy backend instances as Docker containers
    * Terminate containers to simulate scale-down
  - Maintain a registry of active backend instances, such as:
    * A configuration file
    * A database table
  - Ensure the load balancer dynamically updates its backend pool when the registry changes
  - If a cloud testbed is available, optionally integrate with real infrastructure APIs (AWS EC2 via Boto3) to manage instances

- **End-to-End Control Loop Integration**
  - Validate the full control loop:

    $$\text{Telemetry} \rightarrow \text{Forecast} \rightarrow \text{Autoscale} \rightarrow \text{New Resources} \rightarrow \text{Telemetry}$$

  - Simulate rising load by increasing request rates to the load balancer using a load generation script
  - Verify that:
    * The forecasting service predicts the upward trend
    * The autoscaler proactively provisions additional instances

        * The load balancer routes traffic to newly added instances
    - Similarly, test sustained low-load scenarios to validate scale-down behavior

- **Autoscaling Safety Constraints**
    - Introduce basic policy constraints to prevent unsafe scaling decisions
    - Enforce:
        * Maximum number of backend instances (cost control)
        * Minimum number of instances (fault tolerance)
    - Hard-code constraints initially or load them from configuration files
    - Treat these as a precursor to the full Policy Manager in Sprint 8

Sprint Deliverables By the end of Sprint 6, SmartLoad will include:

- A functional forecasting microservice producing short-term load predictions
- An autoscaling controller that proactively adjusts capacity
- Infrastructure hooks (simulated or real) for adding and removing backend instances
- A verified end-to-end control loop integrating prediction and actuation

The sprint demonstration will show that SmartLoad scales resources **proactively**, rather than reactively, reducing response lag to demand spikes and completing the **Control & Actuation layer** of the system architecture.

Sprint 7: Reinforcement Learning Engine Integration (May 1–May 14)

Sprint Goal Integrate a reinforcement learning (RL) decision engine into SmartLoad to enable adaptive, data-driven routing decisions while preserving system safety through staged deployment and fallback mechanisms.

Sprint Tasks

- **Reinforcement Learning Decision Engine Implementation**
    - Implement the RL-based routing engine as a standalone module
    - Use Python with an RL framework (Stable Baselines3 with PyTorch)
    - Define the RL formulation:

        * **State:** current load, per-server latency, server health flags from the anomaly detector
        * **Action:** select a backend server or output a ranked distribution of servers
        * **Reward:** encourage low latency and balanced load (negative P95 latency, throughput–latency trade-off)
    - Start with a simple RL algorithm to validate end-to-end learning, such as:
        * Policy gradient (REINFORCE), or
        * Q-learning in a discrete action space

- **Offline Training in Simulation Environment (Student 2)**
    - Train the RL agent offline using the simulation environment developed in Sprint 4
    - Use CloudSim or a custom simulator to run many training episodes without affecting the live system
    - Simulate long-duration workloads in accelerated time
    - Monitor training progress using reward curves and policy behavior
    - Verify convergence toward reasonable routing behavior (favoring faster or less loaded servers)
    - Save trained model parameters for deployment

- **Shadow Deployment of RL Engine**
    - Integrate the RL engine into the live SmartLoad system in **shadow mode**
    - Allow the RL engine to:
        * Consume live telemetry
        * Compute routing recommendations
    - Log RL decisions without enforcing them, while the load balancer continues using classical routing
    - Compare RL-recommended actions with actual routing decisions to validate correctness and safety
    - Collect performance data under test loads or controlled scenarios

- **Full RL Integration with Routing Logic**
    - Enable RL-driven routing decisions in the load balancer once sufficient confidence is established
    - Implement a clear decision hierarchy:
        * **(1)** Apply anomaly detection constraints to remove unhealthy nodes
        * **(2)** Use RL-generated routing scores or rankings

        * **(3)** Fall back to classical load balancing if RL output is unavailable, uncertain, or warming up
    - Allow policy overrides (safe mode) to bypass RL decisions when required
    - Implement RL communication via the control bus, for example:
        * Publish server rankings or traffic distributions
        * Load balancer consumes RL messages and routes accordingly

- **Online Learning Strategy & Safety Controls**
    - Decide between:
        * Fixed-policy deployment trained offline, or
        * Controlled online learning with continual updates
    - If online learning is enabled:
        * Limit exploration to avoid unsafe routing behavior
        * Gate exploration parameters through configuration or policy constraints
    - Favor conservative defaults suitable for short-term project safety

- **Evaluation of RL-Based Routing**
    - Design experiments to evaluate RL effectiveness, including:
        * Scenarios with one consistently slower backend server
        * Time-varying workloads and traffic spikes
    - Compare RL-based routing against classical algorithms by measuring:
        * Average latency
        * P95 latency
        * Load distribution fairness
    - Use telemetry data to quantitatively assess performance gains

Sprint Deliverables By the end of Sprint 7, SmartLoad will include:

- An integrated reinforcement learning routing engine
- Offline-trained RL policies validated in simulation
- Shadow-mode and production-ready RL integration with fallbacks
- Quantitative evidence comparing RL-based routing to classical strategies

The sprint demonstration will show that under selected conditions, RL-guided routing outperforms static strategies while maintaining fail-safe behavior. At this stage, the **Decision Intelligence Layer**—combining anomaly detection, forecasting, autoscaling, and RL—is fully operational.

Sprint 8: Policy Management & Final Integration (May 15–May 28)

Sprint Goal Finalize SmartLoad by introducing centralized policy management, completing full system integration, and validating correctness, robustness, and performance across realistic end-to-end scenarios.

Sprint Tasks

- **Policy & Configuration Manager Implementation**
  - Develop a lightweight Policy Manager to centralize configuration and high-level control
  - Implement policy distribution using one of the following:
    - Shared JSON/YAML configuration files, or
    - A small configuration service exposing a policy API
  - Support core policy categories, including:
    - **Operating Mode:** classical-only, hybrid (RL + classical), or learning-only
    - **Safe Mode:** force classical routing and disable RL outputs
    - **Scaling Limits:** minimum and maximum number of instances
    - **SLA Targets:** desired maximum P95 latency
    - **RL Parameters:** exploration rate, reward toggles, etc.
  - Ensure all SmartLoad components can read updated policies via the control bus or a shared configuration store
- **Security & Configuration Validation**
  - Ensure sensitive configuration values (cloud API keys) are not hard-coded
  - Store secrets securely using environment variables
  - Add basic validation to detect invalid or unsafe policy values
- **End-to-End Integration Testing**
  - Conduct comprehensive integration tests across all system components
  - Validate the following end-to-end scenarios:
    - **Scenario 1:** steady load → node anomaly → traffic rerouted → recovery and reintegration
    - **Scenario 2:** sudden load spike → forecast predicts increase → autoscaler provisions new instance → RL routes traffic → latency improves
  - Test scale-down behavior after load decreases and verify grace periods

- **Failure Handling & Fallback Validation**
  - Simulate partial system failures, including:
    - RL engine crash or unavailability
    - Metrics database outage
  - Verify that the system continues operating safely using classical load balancing and default behavior
  - Ensure components degrade gracefully using cached or last-known data
- **Resource Usage Monitoring & Optimization**
  - Monitor CPU and memory usage of each SmartLoad component under load
  - Identify and optimize any performance bottlenecks
  - Ensure the overall system remains lightweight and efficient
- **Performance Tuning & Parameter Refinement**
  - Tune anomaly detection thresholds to balance sensitivity and stability
  - Refine forecasting frequency and model parameters for accuracy and efficiency
  - Adjust RL hyperparameters (learning rate, exploration) and retrain models if improvements are observed
  - Optimize the load balancer implementation for throughput (Go concurrency or Node.js clustering)
- **Optional UI / Dashboard Integration (Stretch Goal)**
  - Implement a simple web UI or CLI for operators if time permits
  - Display real-time system status, including:
    - Current load
    - Active backend instances
    - Detected anomalies
  - Allow toggling key policies (enabling or disabling safe mode)
  - If implementation time is limited, prepare static dashboards or logs (Grafana) for demonstration purposes

Sprint Deliverables By the end of Sprint 8, SmartLoad will be:

- Feature-complete, with all architectural layers fully integrated
- Governed by a centralized policy and configuration framework

- Robust to component failures through fallback mechanisms
- Tuned for performance, stability, and efficiency

The final sprint demonstration will showcase SmartLoad handling diverse operational scenarios end-to-end, marking the transition from active development to system evaluation, benchmarking, and project finalization.

Sprint 9: System Testing & Performance Benchmarking (Jun 1–Jun 14)

Sprint Goal Validate SmartLoad's correctness, robustness, and performance through automated testing, fault-injection experiments, and quantitative benchmarking against key system KPIs.

Sprint Tasks

- **Automated Test Suite Development**
  - Develop a comprehensive automated testing framework covering unit, integration, and regression tests
  - Integrate tests into the CI pipeline to ensure repeatability and long-term reliability
- **Unit Testing of Individual Modules**
  - Complete unit tests for any remaining components, including:
    * Anomaly detection logic (flagging known anomalous sequences)
    * Forecasting module outputs on known patterns
    * RL agent decision logic on mock system states
    * Load balancer routing logic and fallbacks
  - Responsibility split by module ownership:
    * ML-related modules (forecasting, anomaly detection, RL)
    * load balancer and routing logic
- **End-to-End Integration Testing (Student 3)**
  - Develop integration test scripts that bring up the full SmartLoad system (via Docker Compose)
  - Simulate realistic scenarios such as:
    * Gradually increasing request load
    * Autoscaler-triggered instance provisioning
    * RL-guided routing under varying load

  - Validate outcomes using assertions, for example:
    * Number of active backend instances increases as expected
    * P95 latency remains below configured SLA targets
  - Implement tests using Python scripts or a testing framework

- **Regression Testing**
  - Identify bugs or edge cases discovered during integration
  - Add targeted regression tests to prevent recurrence

- **Performance Benchmarking**
  - Use a load generation tool (Locust, JMeter, or a custom multithreaded script) to stress the system
  - Gradually increase load until system capacity is reached
  - Measure key performance indicators, including:
    * Throughput (requests per second)
    * Latency distribution (average and P95 latency)
  - Benchmark alternative configurations, such as:
    * RL-based routing vs classical round-robin routing
    * Forecasting-based autoscaling vs reactive autoscaling
  - Measure SmartLoad's own resource consumption (CPU and memory usage per microservice) to assess system overhead

- **Model Accuracy Evaluation**
  - Evaluate forecasting accuracy by comparing predicted versus actual load trends
  - Assess anomaly detection performance using labeled datasets or known injected anomalies
  - Report false positive and false negative rates where applicable

- **Fault Tolerance & Resilience Testing**
  - Intentionally disrupt system components to validate graceful degradation, including:
    * Terminating the RL engine and verifying fallback to classical routing
    * Disabling the forecasting service and confirming autoscaling continues in reactive mode
  - Ensure no system-wide crashes occur and default behavior maintains service availability

- **System Tuning Based on Results**

  - Analyze benchmarking results against SLA targets
  - If P95 latency exceeds acceptable bounds:
    * Adjust autoscaling thresholds or instance limits
    * Optimize load balancer code paths
  - Refine ML components as needed, such as:
    * Retraining RL models with additional data
    * Adjusting reward functions or forecasting parameters
  - Ensure compliance with any explicit performance or efficiency requirements specified by supervisors

- **Results Documentation & Reporting**
  - Collect logs, metrics, and experiment outputs
  - Generate plots and charts for key results, such as:
    * Latency vs load curves
    * Scaling behavior over time
  - Tabulate KPIs comparing baseline behavior with SmartLoad-enabled behavior for inclusion in the final report

Sprint Deliverables By the end of Sprint 9, the team will have:

- A comprehensive automated test suite integrated into CI
- Quantitative performance benchmarks validating system behavior
- Documented evidence of SmartLoad's robustness, scalability, and efficiency

The sprint review will present empirical results demonstrating that SmartLoad meets its design goals, such as maintaining a P95 latency of X ms under Y requests/sec with RL and predictive autoscaling enabled, thereby solidifying confidence in the system's correctness and performance.

Sprint 10: Deployment, Final Documentation & Presentation (Jun 15–Jun 30)

Sprint Goal Finalize SmartLoad for submission by deploying the system in a realistic environment, completing all documentation, and preparing the final report and presentation.

Sprint Tasks

- **Production / Demo Deployment Setup**
  - Prepare a realistic deployment environment for demonstration
  - Deploy SmartLoad using one of the following approaches:
    * Cloud-based virtual machines (AWS, Azure, or university cluster)
    * Kubernetes cluster with SmartLoad components as services
  - Assign roles within the deployment:
    * Load balancer deployed as a public-facing service
    * Backend services deployed as separate instances or pods
  - Configure networking to ensure:
    * Load balancer endpoint is externally accessible
    * Internal communication with backend services is functional
  - Harden deployment artifacts:
    * Production-grade Docker Compose configuration, or
    * Helm charts if using Kubernetes
  - Ensure all environment variables (database connections, service endpoints) are correctly parameterized
- **Final Code Cleanup & Refactoring**
  - Refactor code for clarity and maintainability
  - Remove or disable debug-only logging
  - Ensure all configuration values are externally configurable
  - Eliminate hard-coded constants and magic values
- **Documentation Completion**
  - Consolidate documentation produced across all sprints
  - Ensure consistency between design documents and final implementation
- **Sprint Reports Compilation**
  - Merge sprint-level notes into a cohesive development narrative
  - For each sprint, summarize:
    * Objectives
    * Key implementation outcomes
    * Deviations from the original plan

- **User Guide Preparation**
  - Write a concise guide explaining how to:
    * Deploy and start SmartLoad services
    * Reproduce key test or demo scenarios
    * Interpret logs, metrics, and outputs
  - Include the guide in the repository README or as a standalone markdown document
- **Technical Documentation Finalization**
  - Ensure each SmartLoad module includes:
    * A brief functional description
    * Interface definitions
    * Key design decisions
  - Update existing module design documents (Anomaly Detection, Decision Layer) to reflect the final system state
- **Final Project Report Writing**
  - Prepare the final written report following the expected academic structure:
    * Introduction and problem statement
    * Related work
    * System architecture and design rationale
    * Implementation details
    * Testing and performance evaluation
    * Conclusions and future work
  - Divide drafting responsibilities:
    * introduction and architecture
    * AI/ML components and experimental results
    * infrastructure, deployment, and testing
  - Collaboratively edit and refine the report for coherence and quality
- **Presentation & Demo Preparation**
  - Prepare presentation slides summarizing:
    * Motivation and system overview
    * Key technical contributions
    * Experimental results and insights
  - Design a live demo or scripted demonstration showcasing:
    * Load scaling behavior

        - Anomaly handling
        - RL-based adaptive routing
    - Rehearse the presentation and demo to ensure smooth execution
- **Final Review & Buffer Time**
    - Reserve time for last-minute fixes or refinements
    - Address feedback from supervisors, if any
    - Verify that all deliverables meet submission requirements

Sprint Deliverables By the end of Sprint 10:

- SmartLoad will be fully deployed in a realistic environment
- All documentation (code, user guide, technical docs) will be complete
- The final project report and presentation will be ready for submission

This sprint marks the successful completion of the SmartLoad project, culminating in a fully functional prototype supported by comprehensive documentation, evaluation results, and a polished presentation.